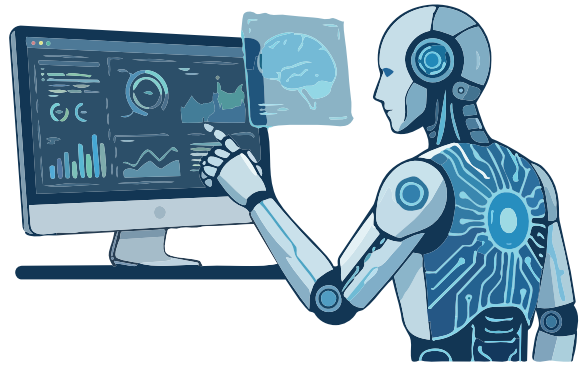
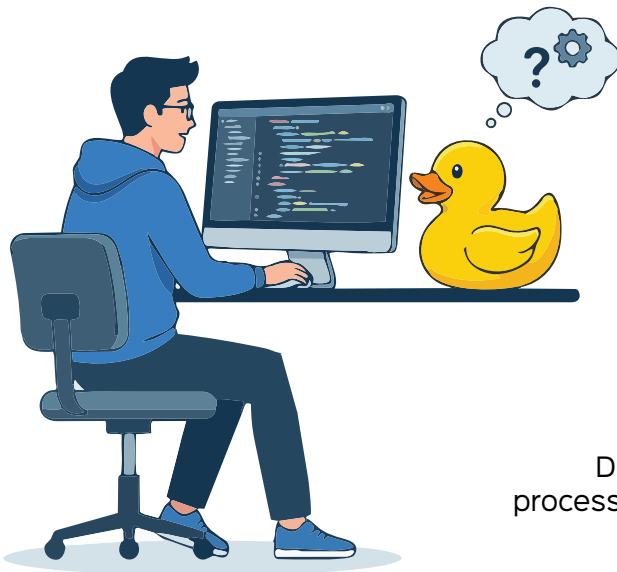


# The Relevance of Rubber Duck Debugging in the Age of AI



Discover why rubber duck debugging is a powerful process today. There's also a step-by-step guide on how to use it in the age of artificial intelligence.

In an era of sophisticated AI coding assistants, the humble practice of rubber duck debugging is gaining new relevance. While an AI can instantly generate code or identify syntax errors, it often lacks the nuanced context of the programmer's original intent. Thus, the 'duck' evolves from a simple passive listener into an active, querying partner; the core magic lies not in receiving an answer, but in the act of formulating the question with such clarity that the solution often reveals itself in the process.

Every engineer has experienced this moment.

You are stuck on a problem for hours. The logs look fine. The logic seems correct. You run the program again and again. Still nothing changes. Finally, you turn to a colleague and begin explaining:

"See, first this function validates the input. Then it calls the API. After that—"  
And suddenly you pause.

"Wait... that API call can fail before validation completes."

The bug was not hiding in the code. It was hiding in your thinking. This phenomenon has a name: rubber duck debugging.

In today's world of AI-powered tools, this simple technique has become even more powerful — but only when used correctly.

## Where did rubber duck debugging come from?

The term became popular after the 1999 book 'The Pragmatic Programmer' by Andrew Hunt and David Thomas. In the book, the authors describe a programmer who keeps a rubber duck on his desk. Whenever he gets stuck, he explains his code line by line to the duck. Quite often, the solution appears during the explanation itself.

But this behaviour existed long before the book. Developers would figure out the problem or the bug while:

- Writing a detailed email about a production issue — and solving it while writing;
- Explaining logic on a whiteboard —

and spotting the flaw mid-sentence;

- Discussing a bug during a stand-up meeting — and realising the mistake while speaking.

The duck is only symbolic. The real idea is simple: When you explain clearly, you think clearly.

## Why does this technique work?

Software development is not just typing code. It is structured thinking. Rubber duck debugging works because it improves that thinking.

**It forces sequential thinking:** Inside our mind, logic feels complete. But when we explain it, we must go step by step:

- What is the input?
- What happens next?
- Why is this condition needed?
- What happens if it fails?

Many developers mentally skip transitions. While explaining, those hidden gaps become visible.

**It exposes assumptions:** Many real-world bugs are based on assumptions:

- This value will never be null.
- The API always returns valid JSON.